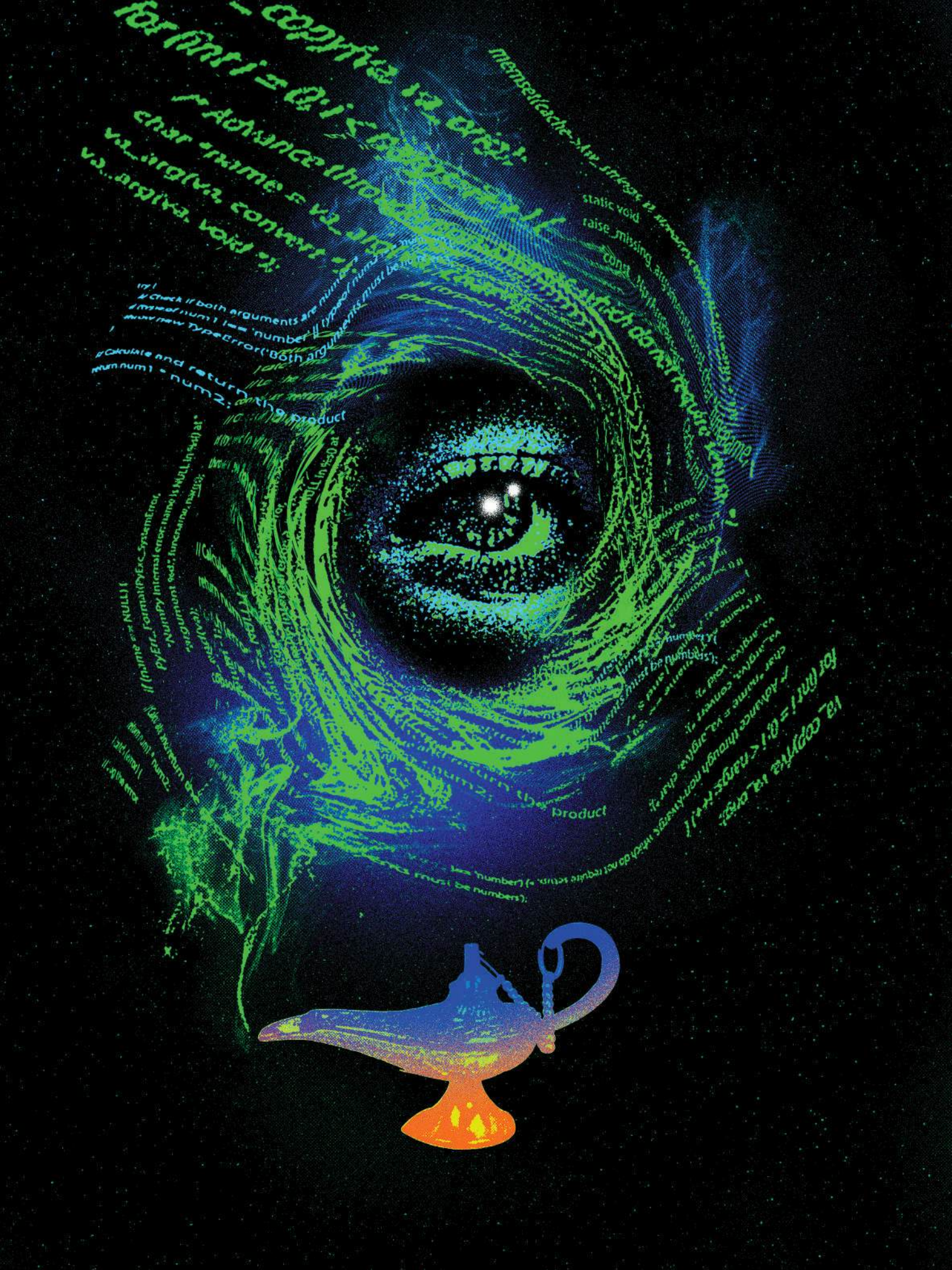


A CRITICAL LOOK at AI-Generated Software

Coding with the new AI tools is both irresistible and dangerous

By
JAIDEEP VAIDYA
& HAFIZ ASIF

Illustrations by Daniel Zender

Authorized licensed use limited to: National Tsing Hua Univ.. Downloaded on February 21,2024 at 14:14:19 UTC from IEEE Xplore. Restrictions apply.

IN MANY WAYS, we live in the world of *The Matrix*. If Neo were to help us peel back the layers, we would find code all around us. Indeed, modern society runs on code: Whether you buy something online or in a store, check out a book at the library, fill a prescription, file your taxes, or drive your car, you are most probably interacting with a system that is powered by software.

And the ubiquity, scale, and complexity of all that code just keeps increasing, with billions of lines of code being written every year. The programmers who hammer out that code tend to be overburdened, and their first attempt at constructing the needed software is almost always fragile or buggy—and so is their second and sometimes even the final version. It may fail unexpectedly, have unanticipated consequences, or be vulnerable to attack, sometimes resulting in immense damage.

Consider just a few of the more well-known software failures of the past two decades. In 2005, faulty software for the US $176 million baggage-handling system at Denver International Airport forced the whole thing to be scrapped. A software bug in the trading system of the Nasdaq stock exchange caused it to halt trading for several hours in 2013, at an economic cost that is impossible to calculate. And in 2019, a software flaw was discovered in an insulin pump that could allow hackers to remotely control it and deliver incorrect insulin doses to patients. Thankfully, nobody actually suffered such a fate.

These incidents made headlines, but they aren't just rare exceptions. Software failures are all too common, as are security vulnerabilities. Veracode's most recent survey on software security, covering the last 12 months, found that about three-quarters of the applications examined contained at least one security flaw, and nearly one-fifth had at least one flaw regarded as being of high severity.

What can be done to avoid such pitfalls and more generally to prevent software from failing? An influential 2005 article in *IEEE Spectrum* identified several factors that are still quite relevant. Testing and debugging remain the bread and butter of software reliability and maintenance. Tools such as functional programming, code review, and formal methods can also help to eliminate bugs at the source. Alas, none of these methods has proven absolutely effective, and in any case they are not used consistently. So problems continue to mount.

Meanwhile, the ongoing AI revolution promises to revamp software development, making it far easier for people to program, debug, and maintain code. GitHub Copilot, built on top of OpenAI Codex, a system that translates natural language to code, can make code recommendations in different programming languages based on the appropriate prompts. And this is not the only such system: Amazon CodeWhisperer, CodeGeeX, GPT-Code-Clippy, Replit Ghostwriter, and Tabnine, among others, also provide AI-powered coding and code completion [see "Robo-Helpers," p. 39].

Most recently, OpenAI launched ChatGPT, a large-language-model chatbot that is capable of writing code with a little prompting in a conversational manner. This makes it accessible to people who have no prior exposure to programming.

ChatGPT, by itself, is just a natural-language interface for the underlying GPT-3 (and now GPT-4) language model. But what's key is that it is a descendant of GPT-3, as is Codex, OpenAI's AI model that translates natural language to code. This same model powers GitHub Copilot, which is used even by professional programmers. This means that ChatGPT, a "conversational AI programmer," can write both simple and impressively complex code in a variety of different programming languages.

This development sparks several important questions. Is AI going to replace human programmers? (Short answer: No, or at least, not immediately.) Is AI-written or AI-assisted code better than the code people write without such aids? (Sometimes yes; sometimes no.) On a more conceptual level, are there any concerns with AI-written code and, in particular, with the use of natural-language systems such as ChatGPT for this purpose? (Yes, there are many, some obvious and some more metaphysical in nature, such as whether the AI involved really understands the code that it produces.)

Authorized licensed use limited to: National Tsing Hua Univ.. Downloaded on February 21,2024 at 14:14:19 UTC from IEEE Xplore. Restrictions apply.

The goal of this article is to look carefully at that last question, to place AI-powered programming in context, and to discuss the potential problems and limitations that go along with it.

While we consider ourselves computer scientists, we do research in a business school, so our perspective here very much reflects on what we see as an industry-shaping trend. Not only do we provide a cautionary message regarding overreliance on AI-based programming tools, but we also discuss a way forward.

WHAT IS AI-POWERED PROGRAMMING?

First, it is important to understand, at least broadly, how these systems work. Large language models are complex neural networks trained on humongous amounts of data—selected from essentially all written text accessible over the Internet. They are typically characterized by a very large number of parameters—many billions or even trillions—whose values are learned by crunching on this enormous set of training data. Through a process called unsupervised learning, large language models automatically learn meaningful representations (known as "embeddings") as well as semantic relationships among short segments of text. Then, given a prompt from a person, they use a probabilistic approach to generate new text.

In its most elemental sense, what the neural network does is use a sequence of words to choose the next word to follow in the sequence, based on the likelihood of finding that particular word next in its training corpus. The neural network doesn't always just choose the most likely word, though. It can also select lower-ranked words, which gives it a degree of randomness—and therefore "interestingness"—as opposed to generating the same thing every time.

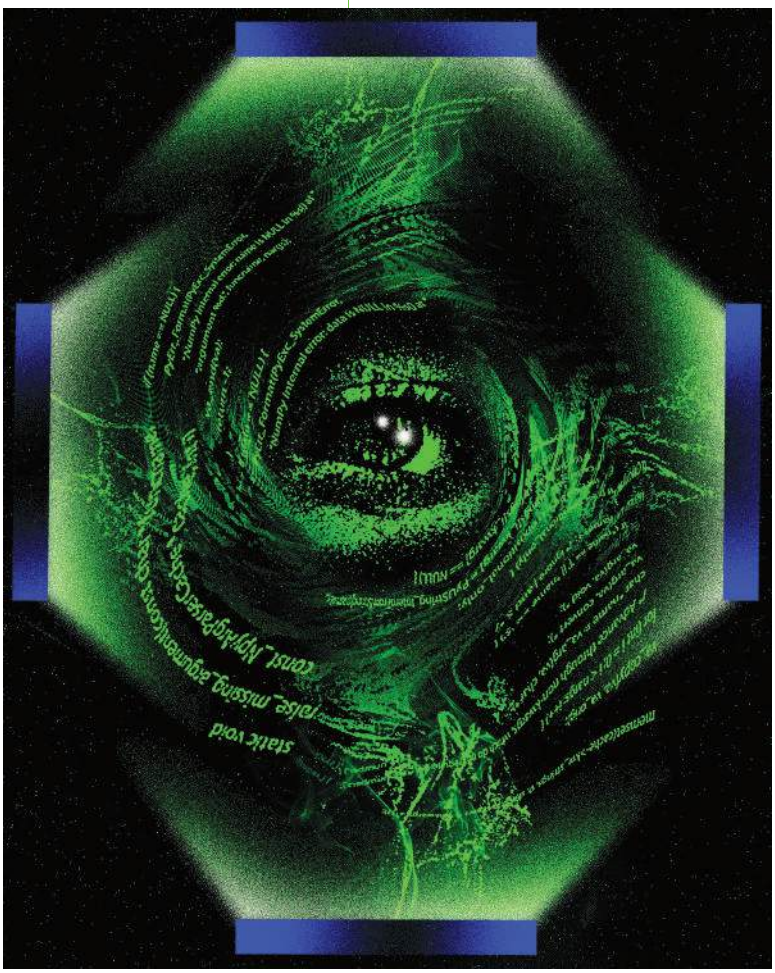
After adding the next word in the sequence, it just needs to rinse and repeat to build longer sequences. In this way, large language models can create very human-looking output, of various forms: stories, poems, tweets, whatever, all of which can appear indistinguishable from the works people produce.

In creating AI tools for generating code, computer programs can themselves be treated as text sequences, with a large language model being trained on code and then used to perform tasks such as code completion, code translation, and even entire programming projects. For example, Codex was trained on a massive dataset of public code repositories, which included billions of lines of code. These models are also fine-tuned to work for specific programming languages or applications, by training the model on a dataset that is specific to the target programming language or type of task at hand.

Even so, the neural network does not have any real understanding of programming, beyond a prescription for how to generate it. So the code that is output can fail on tasks or propagate subtle bugs. One technique these systems use to minimize such issues is to generate a large number of complete programs and then evaluate them against a set of automated tests (the kind many software developers use), providing as output the program that passes the most tests. In any case, these large language models produce code based on what someone has already written—they cannot come up with genuinely new programming solutions on their own.

AYE, ROBOT

Despite the many benefits of AI-powered programming, the use of AI here raises significant concerns, many of which have been pointed out recently by researchers and even by the providers of these AI-based tools themselves. Fundamentally, the problem is this: AI programmers are necessarily limited by the data they were trained on, which includes plenty of bad code along with the good. So the code these systems produce may well have problems, too.



Authorized licensed use limited to: National Tsing Hua Univ.. Downloaded on February 21,2024 at 14:14:19 UTC from IEEE Xplore. Restrictions apply.

BAD PROGRAMMING ADVICE FROM CHATGPT

The AI seems to know about security issues but sometimes misses the most obvious ones

CHATGPT LEARNED ABOUT programming from the code that was available in its massive training set—some of which is good and some of which is bad. So the seemingly authoritative advice it gives can sometimes include code with security vulnerabilities, and you'll need to check it carefully lest you be misled. Here's one illustrative example:

Imagine that you're programming in the Ruby language and want to use a library (which Ruby programmers refer to as a "gem") to perform AES encryption. The Ruby aes gem has encrypt and decrypt functionality, but maybe you want some expert advice on how to use these methods. So you might enter the following prompt into ChatGPT:

PROMPT:

```
aes ruby gem AES.encrypt
```

CHATGPT'S RESPONSE:

Sure, here's an example of how to use the 'AES.encrypt' method from the 'aes' Ruby gem to encrypt data with AES-256 encryption:

```
require 'aes'

key = 'mysecretkey12345'
iv = AES.iv(:AES_256_CBC)
plaintext = 'my secret data'

ciphertext = AES.
encrypt(plaintext, key, iv:
iv)
```

In this example, we generate a random initialization vector (IV) using the 'AES.iv' method with the ':AES_256_CBC' option to specify the CBC mode of operation. We then call the 'AES.encrypt' method with the plaintext data, the key, and the IV to encrypt the data.

To decrypt the encrypted data, you can use the following code:

```
require 'aes'

key = 'mysecretkey12345'
iv = ciphertext[0..15] #
extract the first 16 bytes as
the IV
encrypted_data = cipher-
text[16..-1] # extract the
rest as the encrypted data

plaintext = AES.decrypt(en-
crypted_data, key, iv: iv)
```

In this code, we extract the IV and encrypted data from the ciphertext, and then call the 'AES.decrypt' method with the encrypted data, the key, and the IV to decrypt the data.

Note that the 'AES.encrypt' method uses CBC mode by default, which can be vulnerable to certain types of attacks. If you need to use a more secure mode of operation, such as GCM, you can pass additional options to the 'encrypt' and 'decrypt' methods. For example:

```
require 'aes'

key = 'mysecretkey12345'
iv = AES.iv(:AES_256_GCM)
plaintext = 'my secret data'

ciphertext, tag = AES.
encrypt(plaintext, key, iv:
iv, tag: true)
decrypted_data = AES.
decrypt(ciphertext, key, iv:
iv, tag: tag)
```

In this example, we use the ':AES_256_GCM' option when generating the random IV to specify the GCM mode of operation. We also pass the `tag: true` option to the 'AES.encrypt' method to request the authentication tag, and then pass the tag to the 'AES.decrypt' method to verify the integrity of the decrypted data.

ChatGPT appears authoritative. It even warns about the vulnerability of the default CBC encryption mode to certain kinds of attacks. But ChatGPT missed a greater danger: This gem applies the Ruby hex function on the key. So a programmer using it must take care that the key employed is a string that can be interpreted as a series of hexadecimal numbers. Ruby's hex function applied to the key given in ChatGPT's example code would turn it into zero. Using such a key would result in the encryption being completely insecure.

Not only did ChatGPT fail to include a warning about this drastic vulnerability, its example code could also lead a programmer to fall prey to it. And using additional prompts about key security does little to forestall that danger.

First and foremost are issues with security and reliability. Like the code that people write, AI-produced code can contain all manner of security vulnerabilities. Indeed, a recent research study looked at the result of developing 89 different scenarios for Copilot to complete. Of the 1,689 programs that were produced, approximately 40 percent were found to contain vulnerabilities.

To get a better sense of what we mean by a vulnerability, consider something called a buffer-overflow attack, which takes advantage of the way memory is allocated. In such an attack, a hacker tries to input more data into a buffer (a portion of system memory set aside for storing some particular kind of data) than the buffer can accommodate. What happens next depends on the underlying machine architecture as well as the specific code used. It's possible that the extra data will overflow into adjacent memory and thus corrupt it, which could potentially result in unexpected and perhaps even malicious behavior. With carefully crafted inputs, hackers can use buffer overflows to overwrite system files, inject code, or even gain administrative privileges.

Buffer overflows can be prevented through careful programming practices, such as validating user input and limiting the amount of data that can be placed in a buffer, as well as through architectural safeguards. But there are many other kinds of security vulnerabilities: SQL-injection attacks, improper error handling, insecure cryptographic storage and library use, cross-site scripting, insecure direct object references, and broken authentication or session management, to name just a few common attack strategies. Until there is a way to check for all the different kinds of vulnerabilities and automatically remove them, code generated by an AI system is likely to contain these weaknesses.

A more fundamental problem is that there aren't yet ways to formally specify requirements and to verify that these requirements are met. So it's currently impossible to know that the behavior of an AI-generated program matches what it's supposed to do. A related issue is that the code these AI tools produce is not necessarily optimized for any particular attribute, such as scalability. While it may be possible to achieve that with the right prompts, this brings up the question of how to compose such prompts.

Authorized licensed use limited to: National Tsing Hua Univ.. Downloaded on February 21,2024 at 14:14:19 UTC from IEEE Xplore. Restrictions apply.

Of course, many of these problems exist with the code people write as well. So why should AI-generated code be held to a higher standard?

There are three reasons. First, because the training process utilizes the body of all publicly accessible code, and because there are no straightforward criteria for judging quality, you just don't know how good the code you get from an AI programmer is. The second reason involves psychology. People are apt to believe that computer-generated code will be free of problems, so they may scrutinize it less. And third, because the people using these tools did not create the code themselves, they may not have the skills to debug or optimize it.

There are other thorny issues to consider, too. One is bias, which is insidious: Why did the AI programmer adopt a particular solution when there were multiple possibilities? And what if the approach it adopted is not the best for your application?

Even more problematic are concerns about intellectual property and liability. The data that these models are trained on is often copyrighted. Several legal scholars have argued that the training itself constitutes fair use, but the output of these models may nevertheless infringe on copyrights or violate license terms in the training set. This is particularly relevant because large models can, in many cases, memorize significant parts of the data they are trained on. While there is some very recent work on provable copyright protection for generative models, this area requires significantly more consideration, especially when the notion of a software bill of materials is in the air.

ROBO-HELPERS

This compilation shows the proliferation of AI-based coding assistants

AI Dev Codes	Code GPT	GPT95	Raycast
AI Query	Code Snippets	Hacker AI	Refact
AirOps AI Data	AI	IntelliCode	Refraction AI
Sidekick	CodeAssist	IntelliSense	Replit
AIWriter	Codefy.ai	Jedi	Safurai
aiXcoder	CodeGeeX	JetBrains	Second
AlphaCode	CodeGen	Datalore	SinCode AI
Amazon	Codeium	K.Explorer	Snappify
CodeWhisperer	CodeSquire	Kite	SourceAI
AskCodi	CodeWP	Kodezi	Sourcegraph
Autocode	CometCore	Krater.ai	Cody
Bard	Denigma	Lightly	SpellBox
Bito	DevBox	Marve Chat	StarCoder
Blackbox AI	DevKit	MutableAI	Stenography
Bloop	ExplainDev	Noya	Tabnine
BotCity	Fig	OpenAI Codex	Vivid
Buildt	Figstack	Phind	Warp AI
ChatGPT	Ghostwriter	Programming Helper	What The Diff
ChatGPT Plus	GitFluence	ProMindGPT	Wing Python IDE
Cmd J	GitGab	Q	YouChat
Codacy	GitHub Copilot		Zentask

SOURCE: SOURCEFORGE

PANDORA'S BLACK BOX

Clearly, using any type of automated programming has its dangers. But when these tools are combined with a conversational interface like ChatGPT, the problems are that much more acute. Unlike the AI tools that are primarily used by professional programmers, who should be aware of their limitations, ChatGPT is accessible to everyone. Even novice programmers can use it as a starting point and accomplish quite a lot.

To get a better sense of what is possible, we, along with many others, have asked ChatGPT to answer some common coding questions posed at hiring interviews. Those carrying out such an exercise have come to a range of conclusions, but in general the results show ChatGPT to be quite an impressive job candidate.

And even if ChatGPT is unable to solve a problem the way you want the first time, you can use additional prompts to get to the desired solution eventually. That's because ChatGPT is conversational and remembers the chat history. This is an immensely attractive feature, which suggests that ChatGPT and its successors will sooner or later become part of the software supply chain. To some extent, these tools are already becoming part of the teaching of computer science, apparently with some benefits to students learning to program.

We nevertheless worry that increased reliance on such technologies will prevent programmers from learning important details about how their code actually functions. That seems inevitable. After all, most programmers, even seasoned professionals, aren't thinking in terms of bit manipulation or what's going on in the registers of a CPU or GPU. They reason at much higher levels of abstraction. While that's generally a good thing, there's a danger that the programs they write with AI assistance will become black boxes to them.

And as we mentioned, the code that ChatGPT and other AI-based programming aids produce often contains security vulnerabilities. Interestingly, ChatGPT itself is sometimes aware of this, and it is able to remove such vulnerabilities if requested to do so. But you have to ask. Otherwise it may give you the simplest possible code, which could be problematic if it is used without further thought.

So where do we go from here? Large language models create a conundrum for the future of computer programming. While it's easy enough to create a fragment of code to tackle a straightforward task, the development of robust software for complex applications is a tricky art, one that requires significant training and experience. Even as the application of large language models for programming deservedly continues to grow, we can't forget the dangers of its ill-considered use.

In one way, these models remind us of an aphorism often used to describe working with computers: garbage in, garbage out. And there's plenty of garbage in the training sets these models were built from. Yet they are also immensely capable. ChatGPT, Codex, and other large language models are like the proverbial genie of the lamp, who has the power to give you almost anything you might want. Just be careful what you wish for. ■

Authorized licensed use limited to: National Tsing Hua Univ.. Downloaded on February 21,2024 at 14:14:19 UTC from IEEE Xplore. Restrictions apply.